

University of Wah

Department of Computer Science



Submit to:

Ms. Hassan Shafaqat

Submit by:

Muhammad Talha

Course:

Artificial Intelligence Lab

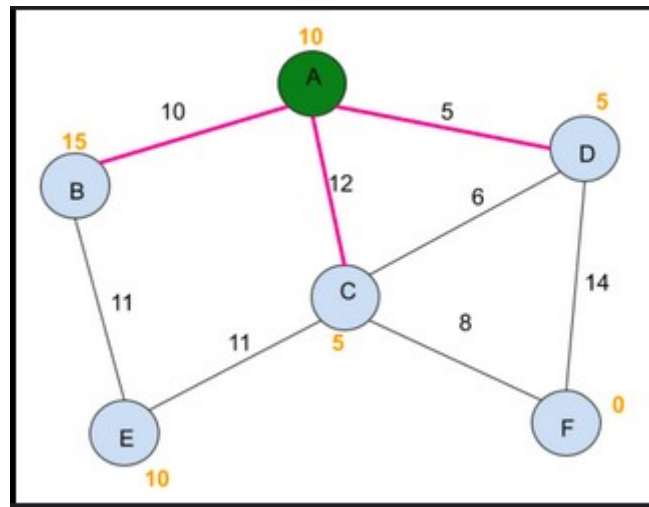
Reg No:

UW-24-CS-BS-101

Section:

BS-CS-3rd-B

Task Graph



Task Code

```
import heapq

graph = {
    'A': [('B', 10), ('D', 5), ('C', 12)],
    'B': [('A', 10), ('E', 11)],
    'C': [('A', 12), ('D', 6), ('E', 11), ('F', 8)],
    'D': [('A', 5), ('C', 6), ('F', 14)],
    'E': [('B', 11), ('C', 11)],
    'F': [('C', 8), ('D', 14)]
}

heuristic = {
    'A': 10,
    'B': 15,
    'C': 5,
    'D': 5,
    'E': 10,
    'F': 0
}

def a_star(start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))

    g_cost = {node: float('inf') for node in graph}
    g_cost[start] = 0

    parent = {}
```

```

while open_list:
    current_f, current = heapq.heappop(open_list)

    if current == goal:
        path = []
        while current in parent:
            path.append(current)
            current = parent[current]
        path.append(start)
        return path[::-1], g_cost[goal]

    for neighbor, cost in graph[current]:
        tentative_g = g_cost[current] + cost

        if tentative_g < g_cost[neighbor]:
            parent[neighbor] = current
            g_cost[neighbor] = tentative_g
            f_cost = tentative_g + heuristic[neighbor]
            heapq.heappush(open_list, (f_cost, neighbor))

    return None, float('inf')

start_node = 'A'
goal_node = 'F'

path, cost = a_star(start_node, goal_node)

print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)

```

Output

```

Shortest Path: A -> D -> F
Total Cost: 19

```